

Condiciones generales

- Cliente: HTML, CSS y JavaScript nativos. No se permiten frameworks ni librerías de terceros (React, Vue, Angular, jQuery, Bootstrap u otros).
- Servidor: PHP sin framework (no Laravel, Symfony, CodeIgniter, etc.). Se permiten librerías puntuales si justificas su uso (en Readme.md).
- Base de datos: PostgreSQL o SQL.
- Puedes consultar documentación, manuales e internet libremente. Puedes usar asistentes de IA.
- No hace falta ningún trabajo de diseño gráfico. No se valora el acabado visual.
- Si no llegas a implementar algún requisito, descríbelo por escrito: explicar correctamente cómo se resolvería, si se deja en blanco contara como No Realizado.

Ejercicio 1

Contexto

En la aplicación, el usuario navega entre secciones (inicio, listados, etc.) sin que el navegador llegue nunca a recargar la página ni a pedir un documento completo nuevo. Lo único que cambia es el contenido dentro de la etiqueta de `<body>`, el `<header>` y `<footer>` se mantienen igual.

Tu tarea es construir el componente de JavaScript que hace posible ese comportamiento.

Requisitos

- Debe existir una única página real. Las zonas comunes (cabecera, menú de navegación y pie) se construyen una sola vez al entrar y no se vuelven a cargar, ni parpadean, al navegar entre secciones.
- Cada sección vive en su propio archivo HTML independiente, que contiene únicamente el fragmento correspondiente a esa sección, no una página completa.
- Al activar un enlace del menú, solo se sustituye el contenido de la zona central por el fragmento de la sección solicitada.
- Debe ser posible componer una misma vista a partir de varios de esos archivos (por ejemplo: contenido principal + panel lateral + un bloque reutilizable que aparezca en varias secciones), de forma que el usuario perciba una sola página.
- Debe estar resuelto qué ocurre cuando se pide una sección que no existe y cuando falla la carga de un fragmento. El usuario nunca debe quedarse ante una pantalla en blanco.
- No se debe volver a pedir al servidor un recurso que ya se tiene sin necesidad. Explica la estrategia que has seguido y en qué casos conviene forzar una recarga.
- Los fragmentos pueden necesitar su propio JavaScript y sus propios estilos. Explica en el README cómo lo harías, cómo evitas que se dupliquen o se pisen entre secciones, y qué pasa con los eventos y los temporizadores de una sección al abandonarla.

Ejercicio 2

Contexto

En una aplicación con decenas de archivos PHP, repetir en cada uno las comprobaciones de sesión, permisos y validación de datos es una fuente constante de errores: basta con que un desarrollador olvide incluirlas en un archivo nuevo para abrir un agujero. La aplicación resuelve esto haciendo que absolutamente todas las peticiones al servidor entren por un único archivo PHP, que centraliza la seguridad, decide qué lógica se ejecuta y se encarga de gestionar y proporcionar más información de los errores.

Tu tarea es construir ese archivo, que permita que se invoquen otros archivos PHP a través de él.

Requisitos

- A partir de lo que recibe en la petición, el archivo debe determinar qué acción hay que ejecutar y con qué datos, y devolver el resultado. Añadir una acción nueva debe ser inmediato y no debe obligar a modificar la lógica del punto de entrada.
- Las comprobaciones comunes se realizan una sola vez, en el punto de entrada, y no se repiten en cada módulo. Como mínimo debe cubrirse: que la acción solicitada existe y está permitida, que hay sesión iniciada, validación y saneado de los datos de entrada y tratamiento centralizado de errores y excepciones.
- Los errores se devolverán en un json y se mostrara con claro detalle al menos en la línea donde se encuentra el error.
- Las respuestas deben tener un formato uniforme y previsible para el cliente: el mismo esquema tanto en caso de éxito como de error, con códigos de estado coherentes.

Ejercicio 3

Contexto

Hay que diseñar la base de datos de una aplicación de inventarios. La particularidad es que los campos que describen un producto no son fijos ni se conocen de antemano: cada inventario define los suyos, y el usuario puede crear campos nuevos desde la propia aplicación, sin que un programador tenga que modificar la estructura de la base de datos ni desplegar nada.

Por ejemplo, un inventario «Vehículos» con los campos Matrícula (texto), Fecha de matriculación (fecha), Kilómetros (número), Asegurado (sí/no) y Centro de trabajo (lista de opciones); y otro inventario «Material informático» con Número de serie (texto), Modelo (texto), Fecha de compra (fecha) y Estado (lista de opciones). Los dos deben funcionar sobre exactamente las mismas tablas.

El modelo debe recoger, como mínimo

- Los inventarios que existen.
- Los tipos de campo que existen en el sistema: texto, número, fecha, sí/no, lista de opciones... ampliables en el futuro.
- Los campos creados, cada uno con su tipo y su configuración: si es obligatorio, si tiene valor por defecto, qué opciones admite cuando es una lista, etc.
- Qué campos tiene asignados cada inventario.
- Los productos, cada uno perteneciente a un inventario.
- El valor de cada campo del inventario para cada producto.

Entregables de este ejercicio

- a) Diagrama entidad-relación del modelo, en imagen o PDF (draw.io, dbdiagram.io o herramienta equivalente), con tablas, columnas, claves primarias y ajenas y cardinalidades.
- b) Un único script SQL que cree la base de datos desde cero: tablas, claves, restricciones de unicidad e índices que consideres necesarios, más las inserciones que dejen cargados los tipos de campo y los dos inventarios del ejemplo con tres o cuatro productos cada uno. El script debe ejecutarse de principio a fin sobre una base de datos vacía sin ningún error.
- c) Media página de explicación: cómo has resuelto el almacenamiento de valores de tipos distintos, qué alternativas has descartado y por qué, y qué inconvenientes tiene la tuya en cuanto a rendimiento, integridad de los tipos y validación de los datos.

Formato de entrega

- Repositorio Git o archivo comprimido con todo el código. Esto se deberá compartir o enviar al correo mandrada@imancorp.es.
- Un README con: qué requisitos has completado y cuáles no, y las explicaciones correspondientes de cada ejercicio.
- El diagrama de la base de datos y el script SQL, en una carpeta identificable.